

Java Servlet InitParam 完整教學指南

📖 教學概述

本教學專案深入探討 Java Servlet 中初始化參數 (InitParam) 的兩種配置方法：

1. 傳統 **web.xml** 配置方式
2. 現代 **@WebServlet** 註解方式

透過您提供的程式碼範例：

```
String name = super.getInitParameter("username");
response.setContentType("text/html;charset=utf-8");
response.getWriter().append("<h2>UserName is ").append(name+"</h2>");
```

我們將展示如何在兩種不同的配置方式中實現相同的功能。

🎯 學習目標

完成本教學後，您將能夠：

- ☒ 理解 Servlet InitParam 的用途和重要性
- ☒ 掌握 web.xml 配置 InitParam 的完整語法
- ☒ 熟練使用 @WebServlet 註解配置 InitParam
- ☒ 比較兩種方法的優缺點和適用場景
- ☒ 在實際專案中選擇適當的配置方式
- ☒ 理解 InitParam 與 Context Parameter 的差異
- ☒ 實作安全且健全的參數讀取機制

📁 專案結構

```
servlet-initparam-tutorial/
├── pom.xml                    # Maven 專案配置
├── src/main/java/com/example/servlet/
│   ├── WebXmlInitParamServlet.java    # web.xml 配置範例
│   └── AnnotationInitParamServlet.java # 註解配置範例
├── src/main/webapp/
│   ├── index.html                # 教學導航頁面
│   └── WEB-INF/web.xml           # Web 應用配置
└── README.md                     # 本教學文件
```

🔑 方法一：web.xml 配置方式

配置文件 (web.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="4.0">

  <!-- Servlet 定義 -->
  <servlet>
    <servlet-name>WebXmlInitParamServlet</servlet-name>
    <servlet-class>com.example.servlet.WebXmlInitParamServlet</servlet-class>

    <!-- 初始化參數配置 -->
    <init-param>
      <param-name>username</param-name>
      <param-value>張小明</param-value>
    </init-param>

    <init-param>
      <param-name>email</param-name>
      <param-value>zhang.xiaoming@example.com</param-value>
    </init-param>

    <init-param>
      <param-name>department</param-name>
      <param-value>資訊工程系</param-value>
    </init-param>

    <init-param>
      <param-name>welcomeMessage</param-name>
      <param-value>歡迎使用 web.xml 配置的 InitParam 範例！</param-value>
    </init-param>

    <init-param>
      <param-name>debugMode</param-name>
      <param-value>false</param-value>
    </init-param>

    <!-- 啟動順序 -->
    <load-on-startup>2</load-on-startup>
  </servlet>

  <!-- URL 映射 -->
  <servlet-mapping>
    <servlet-name>WebXmlInitParamServlet</servlet-name>
    <url-pattern>/WebXmlInitParamServlet</url-pattern>
  </servlet-mapping>

</web-app>
```

Java 實作類別

```
package com.example.servlet;
```

```
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

/**
 * WebXmlInitParamServlet - 使用 web.xml 配置 InitParam 的範例
 */
public class WebXmlInitParamServlet extends HttpServlet {

    private String username;
    private String email;
    private String department;
    private String welcomeMessage;
    private String debugMode;

    /**
     * Servlet 初始化方法
     * 在容器載入時執行一次，讀取 web.xml 中的初始化參數
     */
    @Override
    public void init() throws ServletException {
        super.init();

        // 從 web.xml 讀取初始化參數
        username = super.getInitParameter("username");
        email = super.getInitParameter("email");
        department = super.getInitParameter("department");
        welcomeMessage = super.getInitParameter("welcomeMessage");
        debugMode = super.getInitParameter("debugMode");

        // 設定預設值
        if (username == null) username = "Guest";
        if (email == null) email = "unknown@example.com";
        if (department == null) department = "General";
        if (welcomeMessage == null) welcomeMessage = "Welcome!";
        if (debugMode == null) debugMode = "false";

        // 記錄初始化資訊
        log("WebXmlInitParamServlet 初始化完成");
        log("讀取參數 - username: " + username + ", email: " + email);
    }

    /**
     * 處理 GET 請求
     */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // 設定回應類型和編碼
        response.setContentType("text/html;charset=utf-8");
    }
}
```

```
PrintWriter out = response.getWriter();

try {
    // === 您要求的程式碼範例 ===
    String name = super.getInitParameter("username");
    response.setContentType("text/html;charset=utf-8");
    response.getWriter().append("<h2>UserName is ").append(name+"</h2>");
    // === 範例結束 ===

    // 生成完整的 HTML 回應
    generateDetailedResponse(out);

} catch (Exception e) {
    log("發生錯誤", e);
    showError(out, e.getMessage());
} finally {
    out.close();
}

}

/**
 * 生成詳細的 HTML 回應
 */
private void generateDetailedResponse(PrintWriter out) {
    out.println("<!DOCTYPE html>");
    out.println("<html lang='zh-TW'>");
    out.println("<head>");
    out.println("    <meta charset='UTF-8'>");
    out.println("    <title>web.xml InitParam 範例</title>");
    out.println("    <style>");
    out.println("        body { font-family: Arial, '微軟正黑體'; padding: 20px; background: #f5f5f5; }");
    out.println("        .container { max-width: 800px; margin: 0 auto; background: white; padding: 30px; border-radius: 10px; }");
    out.println("        .highlight { background: #e8f5e8; padding: 15px; border-radius: 5px; border-left: 4px solid #28a745; }");
    out.println("        .param-list { background: #f8f9fa; padding: 20px; border-radius: 5px; }");
    out.println("        .param-item { margin: 10px 0; padding: 10px; background: white; border-radius: 3px; }");
    out.println("    </style>");
    out.println("</head>");
    out.println("<body>");
    out.println("    <div class='container'>");
    out.println("        <h1>🔗 web.xml InitParam 範例</h1>");

    // 顯示您要求的輸出格式
    out.println("        <div class='highlight'>");
    out.println("            <h2>UserName is " + escapeHtml(username) + "</h2>");
    out.println("            <p><em>這是您要求的程式碼輸出格式</em></p>");
    out.println("        </div>");
```

```

        // 顯示所有參數
        out.println("        <div class='param-list'>");
        out.println("            <h3>📖 所有初始化參數</h3>");
        out.println("            <div class='param-item'><strong>使用者名稱:
</strong> " + escapeHtml(username) + "</div>");
        out.println("            <div class='param-item'><strong>電子郵件:
</strong> " + escapeHtml(email) + "</div>");
        out.println("            <div class='param-item'><strong>部門:</strong> "
+ escapeHtml(department) + "</div>");
        out.println("            <div class='param-item'><strong>歡迎訊息:
</strong> " + escapeHtml(welcomeMessage) + "</div>");
        out.println("            <div class='param-item'><strong>除錯模式:
</strong> " + escapeHtml(debugMode) + "</div>");
        out.println("        </div>");

        out.println("        <p><a href='index.html'>返回首頁</a> | <a
href='AnnotationInitParamServlet'>查看註解版本</a></p>");
        out.println("    </div>");
        out.println("</body>");
        out.println("</html>");
    }

    /**
     * HTML 轉義函數
     */
    private String escapeHtml(String input) {
        if (input == null) return "";
        return input.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\"", "&quot;");
    }

    /**
     * 錯誤處理
     */
    private void showError(PrintWriter out, String message) {
        out.println("<html><body>");
        out.println("<h2>錯誤</h2>");
        out.println("<p>" + escapeHtml(message) + "</p>");
        out.println("</body></html>");
    }
}

```

📖 方法二：@WebServlet 註解方式

Java 實作類別 (含註解配置)

```

package com.example.servlet;

import javax.servlet.ServletException;

```

```
import javax.servlet.annotation.WebInitParam;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

/**
 * AnnotationInitParamServlet - 使用 @WebServlet 註解配置 InitParam 的範例
 */
@WebServlet(
    name = "AnnotationInitParamServlet",
    urlPatterns = {"/AnnotationInitParamServlet", "/annotation-init"},
    loadOnStartup = 1,
    initParams = {
        @WebInitParam(name = "username", value = "John Doe"),
        @WebInitParam(name = "email", value = "john.doe@example.com"),
        @WebInitParam(name = "department", value = "Computer Science"),
        @WebInitParam(name = "welcomeMessage", value = "歡迎使用註解式 InitParam 範
例!"),
        @WebInitParam(name = "debugMode", value = "true"),
        @WebInitParam(name = "maxConnections", value = "100"),
        @WebInitParam(name = "timeout", value = "30000"),
        @WebInitParam(name = "version", value = "2.0.1")
    }
)
public class AnnotationInitParamServlet extends HttpServlet {

    private String username;
    private String email;
    private String department;
    private String welcomeMessage;
    private String debugMode;

    /**
     * Servlet 初始化方法
     */
    @Override
    public void init() throws ServletException {
        super.init();

        // 從註解讀取初始化參數
        username = super.getInitParameter("username");
        email = super.getInitParameter("email");
        department = super.getInitParameter("department");
        welcomeMessage = super.getInitParameter("welcomeMessage");
        debugMode = super.getInitParameter("debugMode");

        // 設定預設值
        if (username == null) username = "Anonymous";
        if (email == null) email = "anonymous@example.com";
        if (debugMode == null) debugMode = "false";
    }
}
```

```
        log("AnnotationInitParamServlet 初始化完成");
        log("讀取參數 - username: " + username);
    }

    /**
     * 處理 GET 請求
     */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // 設定回應類型和編碼
        response.setContentType("text/html;charset=utf-8");

        PrintWriter out = response.getWriter();

        try {
            // === 您要求的程式碼範例 ===
            String name = super.getInitParameter("username");
            response.setContentType("text/html;charset=utf-8");
            response.getWriter().append("<h2>UserName is ").append(name+"</h2>");
            // === 範例結束 ===

            // 生成完整的 HTML 回應
            generateDetailedResponse(out);

        } catch (Exception e) {
            log("發生錯誤", e);
            showError(out, e.getMessage());
        } finally {
            out.close();
        }
    }

    /**
     * 生成詳細的 HTML 回應
     */
    private void generateDetailedResponse(PrintWriter out) {
        out.println("<!DOCTYPE html>");
        out.println("<html lang='zh-TW'>");
        out.println("<head>");
        out.println("    <meta charset='UTF-8'>");
        out.println("    <title>@WebServlet InitParam 範例</title>");
        out.println("    <style>");
        out.println("        body { font-family: Arial, '微軟正黑體'; padding: 20px; background: #f0f8ff; }");
        out.println("        .container { max-width: 800px; margin: 0 auto; background: white; padding: 30px; border-radius: 10px; }");
        out.println("        .highlight { background: #e3f2fd; padding: 15px; border-radius: 5px; border-left: 4px solid #2196f3; }");
        out.println("        .param-list { background: #f8f9fa; padding: 20px; border-radius: 5px; }");
        out.println("        .param-item { margin: 10px 0; padding: 10px; background: white; border-radius: 3px; }");
    }
}
```

```
/**
 * HTML 轉義函數
```



```

    */
    private String escapeHtml(String input) {
        if (input == null) return "";
        return input.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\"", "&quot;");
    }

    /**
     * 錯誤處理
     */
    private void showError(PrintWriter out, String message) {
        out.println("<html><body>");
        out.println("<h2>錯誤</h2>");
        out.println("<p>" + escapeHtml(message) + "</p>");
        out.println("</body></html>");
    }
}

```

兩種方法深度比較

1. 配置靈活性

方面	web.xml	@WebServlet
外部配置	☑ 支援	✗ 不支援
運行時修改	☑ 可以 (重啟容器)	✗ 需要重新編譯
環境差異化	☑ 容易	✗ 困難
配置集中管理	☑ 所有配置在一處	✗ 分散在各類別

2. 開發體驗

方面	web.xml	@WebServlet
程式碼簡潔性	✗ 需要維護兩處	☑ 配置與實作在一起
類型安全	✗ XML 字串配置	☑ Java 編譯時檢查
IDE 支援	✗ XML 編輯	☑ Java 自動完成
重構友善性	✗ 容易遺漏	☑ IDE 自動重構

3. 團隊協作

方面	web.xml	@WebServlet
衝突機率	✗ 多人修改同一檔案	☑ 各自修改各自類別
責任劃分	☑ 配置與開發分離	✗ 開發者需懂配置

方面	web.xml	@WebServlet
版本控制	✗ 合併衝突較多	✓ 衝突較少
學習曲線	✗ 需要學習 XML	✓ Java 開發者友善

🔗 核心程式碼範例

您要求的基本範例

```
// 兩種方法都使用相同的程式碼來讀取參數
String name = super.getInitParameter("username");
response.setContentType("text/html;charset=utf-8");
response.getWriter().append("<h2>UserName is ").append(name+"</h2>");
```

輸出結果：

- web.xml 版本：UserName is 張小明
- 註解版本：UserName is John Doe

進階參數處理範例

```
@Override
public void init() throws ServletException {
    super.init();

    // 安全的參數讀取
    username = getInitParameter("username");
    if (username == null || username.trim().isEmpty()) {
        username = "Default User";
        log("警告：username 參數未設定，使用預設值");
    }

    // 數字參數處理
    String timeoutStr = getInitParameter("timeout");
    try {
        timeout = Integer.parseInt(timeoutStr);
    } catch (NumberFormatException e) {
        timeout = 30000; // 預設 30 秒
        log("警告：timeout 參數格式錯誤，使用預設值：" + timeout);
    }

    // 布林參數處理
    debugMode = "true".equalsIgnoreCase(getInitParameter("debugMode"));

    log("初始化完成 - username: " + username + ", timeout: " + timeout +
        ", debug: " + debugMode);
}
```

參數驗證和錯誤處理

```
/**
 * 安全地讀取字串參數
 */
private String getStringParam(String name, String defaultValue) {
    String value = getInitParameter(name);
    if (value == null || value.trim().isEmpty()) {
        log("參數 " + name + " 未設定，使用預設值: " + defaultValue);
        return defaultValue;
    }
    return value.trim();
}

/**
 * 安全地讀取整數參數
 */
private int getIntParam(String name, int defaultValue) {
    String value = getInitParameter(name);
    if (value == null || value.trim().isEmpty()) {
        return defaultValue;
    }

    try {
        return Integer.parseInt(value.trim());
    } catch (NumberFormatException e) {
        log("參數 " + name + " 格式錯誤: " + value + "，使用預設值: " +
defaultValue);
        return defaultValue;
    }
}

/**
 * 安全地讀取布林參數
 */
private boolean getBooleanParam(String name, boolean defaultValue) {
    String value = getInitParameter(name);
    if (value == null || value.trim().isEmpty()) {
        return defaultValue;
    }
    return "true".equalsIgnoreCase(value.trim());
}
```



部署和測試指南

1. 編譯和打包

```
# 進入專案目錄
cd servlet-initparam-tutorial
```

```
# 清理並編譯
mvn clean compile

# 打包成 WAR 檔案
mvn package
```

2. 部署到 Tomcat

```
# 複製 WAR 檔案到 Tomcat
cp target/servlet-initparam-tutorial-1.0.0.war $TOMCAT_HOME/webapps/

# 啟動 Tomcat
$TOMCAT_HOME/bin/startup.sh # Linux/Mac
$TOMCAT_HOME/bin/startup.bat # Windows
```

3. 測試訪問

- 主頁導航：<http://localhost:8080/servlet-initparam-tutorial-1.0.0/>
- **web.xml** 範例：<http://localhost:8080/servlet-initparam-tutorial-1.0.0/WebXmlInitParamServlet>
- 註解範例：<http://localhost:8080/servlet-initparam-tutorial-1.0.0/AnnotationInitParamServlet>

4. 預期輸出

訪問任一 Servlet 都會看到：

```
<h2>UserName is [配置的使用者名稱]</h2>
```

以及完整的參數列表和詳細說明。

💡 最佳實務建議

1. 選擇配置方式的準則

使用 **web.xml** 當：

- 企業級應用需要靈活的外部配置
- 需要在不同環境中使用不同參數
- 團隊有專門的配置管理人員
- 使用傳統的應用伺服器部署

使用 **@WebServlet** 當：

- 快速開發和原型設計
- 小型到中型應用

- 團隊成員都是 Java 開發者
- 使用現代的微服務架構

2. 參數命名約定

```
// 推薦的參數命名方式
@WebInitParam(name = "database.url", value = "jdbc:mysql://localhost:3306/app")
@WebInitParam(name = "cache.timeout", value = "3600")
@WebInitParam(name = "feature.debug.enabled", value = "false")
@WebInitParam(name = "ui.theme.default", value = "light")
```

3. 安全性考量

```
// 不要在 InitParam 中儲存敏感資訊
@WebInitParam(name = "database.password", value = "secret123") // ✕ 不好

// 使用外部配置或環境變數
String dbPassword = System.getenv("DB_PASSWORD"); // ✓ 較好
```

4. 效能最佳化

```
@Override
public void init() throws ServletException {
    super.init();

    // 在初始化時讀取所有參數並快取
    this.configCache = new HashMap<>();
    Enumeration<String> paramNames = getInitParameterNames();
    while (paramNames.hasMoreElements()) {
        String name = paramNames.nextElement();
        configCache.put(name, getInitParameter(name));
    }
}

// 在請求處理中使用快取的值
private String getCachedParam(String name) {
    return configCache.get(name);
}
```

🔍 進階主題

1. InitParam vs Context Parameter

```
// InitParam - Servlet 專用
String servletParam = getInitParameter("servletSpecific");
```

```
// Context Parameter - 整個應用程式共用
String appParam = getServletContext().getInitParameter("applicationWide");
```

2. 動態參數更新 (僅 web.xml)

```
// 監控配置檔案變更 (企業級功能)
public class ConfigMonitor implements ServletContextListener {
    @Override
    public void contextInitialized(ServletContextEvent sce) {
        // 啟動配置檔案監控
        startConfigFileWatcher();
    }
}
```

3. 參數加密

```
// 對敏感參數進行加密存儲
@WebInitParam(name = "encrypted.token", value = "AES:encrypted_value_here")

// 在初始化時解密
private String decryptParam(String encryptedValue) {
    if (encryptedValue.startsWith("AES:")) {
        return aesDecrypt(encryptedValue.substring(4));
    }
    return encryptedValue;
}
```

學習檢核清單

完成本教學後，請確認您能夠：

- ☐ 理解 InitParam 的生命週期和作用域
- ☐ 在 web.xml 中正確配置 Servlet 和 InitParam
- ☐ 使用 @WebServlet 和 @WebInitParam 註解
- ☐ 安全地讀取和驗證初始化參數
- ☐ 比較兩種配置方式的優缺點
- ☐ 根據專案需求選擇適當的配置方式
- ☐ 實作健全的錯誤處理機制
- ☐ 理解參數命名和組織的最佳實務

延伸閱讀資源

相關技術

- **Servlet Filter**：請求前置處理

- **ServletContextListener**：應用程式生命週期管理
- **JNDI**：企業級資源配置
- **Spring Boot**：現代 Java Web 開發

推薦學習路徑

1. 掌握基本的 Servlet API
2. 學習 Filter 和 Listener 概念
3. 了解 Spring Framework 的 DI 容器
4. 探索 Spring Boot 的自動配置機制

總結

通過本教學，您已經完整學習了 Java Servlet InitParam 的兩種配置方法。關鍵要點：

1. **web.xml** 配置適合需要外部靈活配置的企業級應用
2. **@WebServlet** 註解適合快速開發和小型專案
3. 兩種方法的程式碼讀取方式完全相同
4. 都需要注意參數驗證和錯誤處理
5. 選擇配置方式要考慮專案規模、團隊結構和部署需求

您提供的程式碼範例：

```
String name = super.getInitParameter("username");
response.setContentType("text/html;charset=utf-8");
response.getWriter().append("<h2>UserName is ").append(name+"</h2>");
```

在兩種配置方式中都能完美運作，這就是 Servlet API 設計的優雅之處！

作者：程式設計教學教授

版本：1.0

最後更新：2025年10月

授權：教育用途免費使用